Learning Path: Skills, MCP and the Firmware Kit, One Lesson at a Time

Oct 10, 2026 · @Abdelrahman Wael

How to use this guide

Thirteen short lessons take you from a bare agent to writing your own skills, subagent and MCP server, using the firmware kit as the worked example. Do them in order; each one builds on files the previous one created. Lessons 0 to 11 run in a throwaway practice repository, so nothing touches work code until lesson 12.

The lessons are written for Gemini CLI, since that is what you have at work, with the kit from `fw-agent-kit-gemini.zip`. A table at the end gives the Claude Code equivalent of each command.

Each lesson has the same shape: what you will learn, the steps, and a check that tells you it worked. Do not move on until the check passes. If something behaves differently from what the lesson says, that is the lesson: find out why before continuing.

Progress

- [ ] Lesson 0: practice repository (20 min)
- [ ] Lesson 1: the bare agent and the instruction file (30 min)
- [ ] Lesson 2: install the kit and read one skill (30 min)
- [ ] Lesson 3: the feedback skills (45 min)
- [ ] Lesson 4: one slice end to end (90 min)
- [ ] Lesson 5: your first skill (45 min)
- [ ] Lesson 6: a script-backed skill and a slash command (60 min)
- [ ] Lesson 7: test and improve a skill (60 min)
- [ ] Lesson 8: guardrails (30 min)
- [ ] Lesson 9: use an MCP server (45 min)
- [ ] Lesson 10: build a tiny MCP server (60 min)
- [ ] Lesson 11: your own subagent (45 min)
- [ ] Lesson 12: move to the real repository (half a day)

Three habits to keep from day one

1. **Start a fresh session for every test.** A session that watched you write a skill already knows what you meant, and will hide the gaps in what you wrote.
2. **Read before you run.** Open every `SKILL.md`, script and config before the agent uses it. That is how you learn the format, and it is the same habit that keeps you safe with other people's skills.
3. **Commit after every lesson.** A bad experiment then costs one `git checkout`.

Lesson 0: the practice repository

You will learn: nothing about agents yet. You are building a safe place to make mistakes.

The practice repository is `fw-practice.zip`: a ring buffer in C with one test, and four small scripts that stand in for a real toolchain. It also has a `gen/` folder playing generated code and a `scripts/flash.sh` that only prints a message, so later lessons can test the guardrails without any risk.

File	Stands in for
`scripts/build.sh <variant>`	Your Green Hills build. Uses `gcc` with warnings as errors
`scripts/test.sh [module]`	Host unit tests
`scripts/lint.sh <files>`	A static checker. Uses extra `gcc` warnings
`scripts/size.sh <variant>`	Section sizes from the map file
`scripts/flash.sh`	A flash tool the agent must never run
`gen/rte_types.h`	Generated code the agent must never edit
`solutions/`	Finished versions of the files you write in lessons 5, 6, 10 and 11

Steps

1. In WSL, check the tools: `gcc --version`, `python3 --version`, `git --version`, `gemini --version`.
2. Unzip and make it a git repository:

```
unzip fw-practice.zip && cd fw-practice
git init && git add -A && git commit -m "practice repo"
```

3. Run each script by hand once:

```
scripts/build.sh dev
scripts/test.sh
scripts/lint.sh src/ringbuf.c
scripts/size.sh dev
```

Check

- [ ] The build prints `Build OK (dev)` and the test prints `All tests passed`.
- [ ] You have read `src/ringbuf.c` and the test. You should know this code better than the agent does.

Do not look in `solutions/` until you have tried the lesson yourself.

Lesson 1: the bare agent and the instruction file

You will learn: what the agent does with no help, and how much one short always-loaded file changes it. This is the baseline everything else is measured against.

Steps

1. Start `gemini` in the practice repository. Make sure there is no `GEMINI.md` yet.
2. Ask: `Add a function ringbuf_peek that returns the oldest value without removing it. Add a test.` Approve its actions and watch what it does. Note down:
   - Did it find and run the build and test scripts, or guess a compiler command?
   - Did it follow the style of the existing code (types, `u` suffixes, brace style)?
   - Did it change anything you did not ask for?
3. Throw the result away: `git checkout . && git clean -fd`.
4. Create `GEMINI.md` by hand. Keep it this short:

```
# Project
Ring buffer library in C99. Practice repository.

# Commands
- Build: scripts/build.sh dev
- Test: scripts/test.sh

# Rules
- Fixed-width types and unsigned literals with a u suffix, as in
src/ringbuf.c.
- Every new function gets a test in tests/.
- Change only what the task needs.
- Never edit anything under gen/. Never run scripts/flash.sh.
```

5. Run `/clear` to start a fresh session, then `/memory show` to confirm the file is loaded. Give the same request as in step 2 and compare with your notes.
6. Commit `GEMINI.md`, and keep or discard the code change as you like.

Check

- [ ] You can name two things the agent did differently the second time.
- [ ] You can explain why the build command belongs in this file and a long procedure does not. (The file is loaded on every turn of every session, so every line costs tokens all the time.)

Try next: `/init` generates a `GEMINI.md` for you. Run it in a scratch copy and compare its output with your hand-written one. Generated files are usually longer than they should be.

Lesson 2: install the kit and read one skill

You will learn: what a skill is made of, where it lives, and what the agent sees of it before and after it is activated.

Steps

1. Install the kit into the practice repository, from a normal terminal:

```
python3 /path/to/fw-agent-kit-gemini/install.py --project .
git status
```

2. Look at what arrived before using any of it:

Path	What it is
`.agents/skills/`	14 skill folders, each with a `SKILL.md`
`.gemini/commands/`	10 small TOML files that become `/vm:...` and `/fw:...`
`.gemini/agents/level-reviewer.md`	The reviewer subagent
`GEMINI.md`	Your file from lesson 1, with the kit's ten hard rules appended as a marked block
`tools/fw-agent/`	The Python scripts the skills call
`.fw-agent.json`, `docs/agent/`	Configuration and reference templates

3. Open `.agents/skills/fw-build/SKILL.md` and find its two parts:
   - The **frontmatter** between the `---` lines: `name` and `description`. The description is the only thing the agent sees at the start of a session. It is the trigger.
   - The **body**: the instructions. The agent reads these only after the skill is activated.
4. Open `.gemini/commands/fw/build.toml`. It is three lines: a description, and a prompt that says "activate the `fw-build` skill". In Gemini CLI a skill is not a command by itself; this file is what makes `/fw:build` exist.
5. Start `gemini`, then run `/skills list` and `/commands list`.
6. Ask, without using any command: `What skills do you have for building and testing firmware?` The answer comes from the descriptions alone.

Check

- [ ] `/skills list` shows 14 skills whose names start with `fw-` or `vm-`.
- [ ] You can say in one sentence what the agent knows about `fw-build` before it is activated, and what it knows after.
- [ ] You have read two more skills of your choice and can say which is a procedure (`vm-slice`) and which is reference knowledge (`fw-mcu`).

If the skills are not listed, check your Gemini CLI version and whether your organisation has switched skills off. Nothing later in this guide works without them.

Lesson 3: the feedback skills

You will learn: how to run a skill on purpose and by description, what activation looks like, and why a fast checker is what makes an agent correct itself.

Steps

1. Run `/fw:setup`. Gemini CLI asks you to approve activating the skill; read the prompt, then approve. The skill finds the four scripts and proposes values for `.fw-agent.json`. Check each proposed command against what you ran by hand in lesson 0. Skip its policy step for now; that is lesson 8.
2. Open `.fw-agent.json` yourself and confirm `protected_paths` contains `gen/**` and `blocked_commands` contains `scripts/flash`.
3. Run `/fw:build`. Then, in a normal terminal, run the wrapper directly and compare:

```
python3 tools/fw-agent/run_step.py build
cat .fw-agent/logs/build.log
```

The skill is instructions; the script does the work. The agent gets the short summary, and the full log stays on disk.

4. Break the build on purpose. In `src/ringbuf.c`, add an unused variable inside `ringbuf_put`. Then say only: `the build is broken, fix it`. You did not type a command, so watch whether it activates `fw-build` from the description.
5. Break a test on purpose. In `tests/test_ringbuf.c`, change `v == 42u` to `v == 43u`. Say: `run the tests`. Read its reasoning: the `fw-unit-test` skill tells it to decide whether the code or the test is wrong, and never to weaken a test. See whether it does.
6. Run `/fw:size-budget` and `/fw:lint-delta` once each to see their output.
7. Restore and commit: `git checkout src tests`, then commit the kit files.

Check

- [ ] In step 4 the agent fixed one error and rebuilt, without rewriting anything else.
- [ ] In step 5 it identified the test as the wrong side and told you, instead of changing `ringbuf_get` to return 43.
- [ ] You can explain the difference between `/fw:build` (you chose the skill) and step 4 (the model chose it).

Experiment: disable the skill with `/skills disable fw-build`, start a fresh session, and repeat step 4. Notice what the agent does without it. Re-enable it afterwards. This with-and-without comparison is the basic test for any skill.

Lesson 4: one slice end to end

You will learn: how a workflow skill chains levels, how a subagent gives an independent review, and where the human gates are. This is the longest lesson; do it in one sitting.

The feature: *the ring buffer reports how many bytes are free, and rejects a null pointer on every function.*

Steps

1. **System level.** Run `/vm:slice ring buffer reports free space and rejects null pointers`. It creates `docs/slices/S001-.../01-system.md` with requirements (`REQ-S001-01` ...) and acceptance tests (`AT-S001-01` ...). Read the file. Is every requirement testable? Did it invent anything you did not say?
2. **Review.** Run `/vm:review S001 system`. The skill runs a script that prepares a review pack, then hands it to the `level-reviewer` subagent, which reads everything cold. Read the findings in `REVIEW.md`.
3. **Your gate.** If you agree with an approve verdict, edit `01-system.md` yourself and change `Status: draft` to `Status: approved`. If the reviewer found problems, ask the agent to fix the ones you agree with and review again.
4. **Test the gate.** Before approving, run `/vm:arch S001` once on purpose. It should refuse, because the system level is not approved. Then approve and run it again.
5. **Architecture level.** `/vm:arch S001` allocates the requirements, writes the new declarations into `src/ringbuf.h`, builds, and adds failing integration test stubs. Check that the header really changed and the build still passes.
6. Review and approve as before: `/vm:review S001 architecture`, then set the status.
7. **Low level.** `/vm:lld S001` designs the units, writes each unit test before its code, then runs build, tests, lint, size and the trace check.
8. Review and approve: `/vm:review S001 lowlevel`.
9. **Trace.** Run `/vm:trace S001`, then run the script yourself:

```
python3 tools/fw-agent/trace_check.py S001 --strict
```

Check

- [ ] In step 4 the architecture skill refused to start on an unapproved level.
- [ ] Each test in `tests/` carries its ID (`UT-S001-..`, `IT-S001-..`) in a name or comment.
- [ ] `trace_check.py --strict` reports no `uncovered`, `dangling` or `empty` gaps.

- [ ] You found at least one thing in a review that you would not have caught, or one finding you disagreed with. Either is a useful result.
- [ ] `git diff --stat` shows changes only in `src/`, `tests/` and `docs/slices/`.

What to notice

- The reviewer has only two tools, reading and searching. Open `.gemini/agents/level-reviewer.md` and find the `tools` list. That list is why it cannot "fix" what it reviews.
- The agent never writes `Status: approved`. Find the rule that says so in `GEMINI.md`.
- If the agent tried to do all three levels in one go, the skill text did not hold it. That is a real finding about the skill, and lesson 7 shows how to fix such things.

Lesson 5: your first skill

You will learn: to write a skill from nothing, and that the description decides whether it ever runs.

You will write a reference skill: knowledge the agent applies while doing something else. The subject is a commit message convention, because it is small and you can check the result at a glance.

Steps

1. Create the folder and file by hand. Do not ask the agent to write it; the point is to learn the format.

```
mkdir -p .agents/skills/commit-style
```

2. Write `.agents/skills/commit-style/SKILL.md`:

```
---
name: commit-style
description: The commit message convention of this repository. Use when
writing a commit message, committing changes, or reviewing whether a commit
message is acceptable.
---

Format: `<type>(<module>): <summary> [<ticket>]`

- type is one of: feat, fix, test, refactor, docs, build.
- module is the file or component touched, for example ringbuf.
- summary is imperative, lower case, at most 60 characters, no full stop.
- ticket is the ID the user gave. If none was given, ask for it; never invent
one.
- One logical change per commit. If the diff mixes two, propose two commits.

Show the message and wait for approval before committing.
```

3. Start a fresh session (`/clear` , or restart) and run `/skills list` . The new skill should be there. If not, run `/skills reload` .
4. **Trigger test.** Make a small change to `src/ringbuf.c` , then try each prompt below in a fresh session and record whether the skill activated.

Prompt	Should it activate?
`commit this change`	Yes
`write a commit message for my diff`	Yes
`is "fixed stuff" a good commit message here?`	Yes
`what does ringbuf_put do?`	No
`build the project`	No
`show me the git log`	No

5. **Break the description.** Change the description to `Commit helper.` and repeat the three "yes" prompts in fresh sessions. Then restore the good description.
6. Commit the skill, using the skill.

Check

- [ ] With the good description, at least the three "yes" prompts activated it and none of the "no" prompts did.
- [ ] With `Commit helper.` it activated less often or not at all. You have now seen why the description is the most important line in a skill.
- [ ] The commit message it produced follows the format, and it asked you for a ticket ID instead of inventing one.

The pattern for a description: what it is, then `Use when` followed by the situations in the words you would actually type. Put the main use first, because long lists get shortened.

Lesson 6: a script-backed skill and a slash command

You will learn: when to put the work in a script instead of in instructions, and how to give a skill an explicit command.

Use a script when a step must give the same answer every time. The task here: check that every header has an include guard named after its file (`ring_buf.h` must use `RING_BUF_H`). An agent can do this by reading files, but slowly and not reliably across a hundred headers. A script does it in milliseconds.

Steps

1. Create the folders:

```
mkdir -p .agents/skills/header-guards/scripts .gemini/commands/fw
```

2. Write the script `.agents/skills/header-guards/scripts/check_guards.py` . Write it yourself or copy it from `solutions/lesson6/` . It takes folders as arguments (default `src`), prints one line per problem and a summary, exits 1 when it finds problems, and explains itself with `--help` .
3. Test the script by hand before any agent sees it:

```
python3 .agents/skills/header-guards/scripts/check_guards.py --help
python3 .agents/skills/header-guards/scripts/check_guards.py src
printf '#ifndef WRONG_H\n#define WRONG_H\n#endif\n' > src/can_if.h
python3 .agents/skills/header-guards/scripts/check_guards.py src
```

4. Write `.agents/skills/header-guards/SKILL.md` :

```
---
name: header-guards
description: Checks that every C header has an include guard named after its
file. Use after adding or renaming a header, when asked to check include
guards, and before a commit that touches header files.
---

1. Run `python3 scripts/check_guards.py --help` in this skill's folder once
to see the options.
2. Run it on the folders that hold headers, `src` by default.
3. For each problem line, fix the guard in that header: both the `#ifndef`
and the `#define`.
4. Never touch headers under `gen/` or `vendor/`. Report them instead.
5. Run the script again and report the summary line.
```

5. Give it a command. Write `.gemini/commands/fw/guards.toml`:

```
description = "Check include guards in C headers"
prompt = """
Activate the `header-guards` skill and follow it exactly. If the skill is not
available, say so and stop.

Request: {{args}}
"""
```

6. Fresh session. Run `/commands list` to see `/fw:guards`, then run it. It should find the wrong guard in `src/can_if.h` and fix it.
7. Fresh session. Create another bad header, then say only: `I added a header, check it`. See whether the description alone triggers the skill.
8. Commit.

Check

- [ ] The script works from the terminal with no agent involved.
- [ ] `/fw:guards` ran the script and did not read the headers one by one to check them itself.
- [ ] It read `--help` instead of opening the script's source. If it read the source, strengthen step 1 of the skill.
- [ ] It left `gen/rte_types.h` alone.

Rule of thumb: instructions for judgment, scripts for facts. If you find yourself writing an algorithm in a skill body, it should be a script.

Lesson 7: test and improve a skill

You will learn: a repeatable loop for making a skill better, and how to tell whether a skill is worth keeping at all.

A skill that activates has only proved the agent found it. It has not proved the skill helped. Test the two separately.

Steps

1. **Write the test sheet.** Create `.agents/skills/header-guards/TESTS.md` with two lists: five prompts that should activate the skill and five near-misses that should not (for example `rename ringbuf.h to ring_buffer.h` should; `what is an include guard?` should not). Add three tasks with the result you expect.
2. **Trigger run.** Try each of the ten prompts in a fresh session and mark hit or miss.
3. **Fix the description, not the prompts.** For each miss, ask which words in your prompt were missing from the description, and add them. For each false hit, narrow the description or say what it is not for. Rerun only the prompts that failed.
4. **Output run, with and without.** Pick one task from the sheet. Run it in a fresh session with the skill enabled, then run `/skills disable header-guards`, start a fresh session and run it again. Compare time, number of steps and correctness. Re-enable the skill.
5. **The failure-to-fix loop.** Use the skills from lessons 5 and 6 for real small tasks. Every time the agent does something you did not want, decide where the fix belongs:

What went wrong	Where the fix goes
The skill did not activate	The description
It activated but skipped a step	Move the step up, number it, say when it applies
It did a step wrongly	Make the step concrete: exact command, exact output expected
It did something extra	Add a "do not" line, or tighten the scope sentence

What went wrong	Where the fix goes
It forgot the rule late in a long session	Phrase it as a standing rule ("after every edit ..."), put it near the top
It must never happen, whatever the wording	Not a skill problem. Use a policy rule (lesson 8)
The same fact is needed in every task	Not a skill. Put it in `GEMINI.md`

6. Make one change at a time and rerun the failing case. If you change three things and it improves, you will not know which one mattered.
7. Apply the loop to one kit skill. If lesson 4 showed the agent running past the end of a level, strengthen the "stop" step in that skill and test again.

Check

- [ ] Your test sheet exists and all ten trigger prompts behave as expected.
- [ ] You have a with-and-without comparison written down for one task, and you can say what the skill bought you.
- [ ] You changed at least one skill because of an observed failure, and the failing case now passes.

When to delete a skill: if the with-and-without runs give the same result, the skill is costing listing space for nothing. Remove it. Most skills that restate what the model already knows about C fall into this group.

Lesson 8: guardrails

You will learn: the difference between advice the model follows and a rule the tool enforces, by testing both.

So far the only thing stopping the agent from editing `gen/` or running `scripts/flash.sh` is text in `GEMINI.md`. That is advice. This lesson adds enforcement through the Gemini CLI policy engine.

Steps

1. **Test the advice first.** In a fresh session say: `add a comment at the top of gen/rte_types.h`. Then: `run scripts/flash.sh to check it works`. Note what happens. With the hard rules loaded it should refuse both, but nothing forces it to.
2. **Look at the rules before applying them.** In a normal terminal:

```
python3 tools/fw-agent/policy.py
```

3. Read the output. Each `[[rule]]` names a tool, a pattern and a decision. Find the rule for `gen/**` and notice that it contains the absolute path of this repository, so it cannot affect your other projects.
4. **Apply them.** This writes one file under `~/.gemini/policies/`:

```
python3 tools/fw-agent/policy.py --apply
```

5. Restart Gemini CLI and run `/policies list` to confirm the rules are loaded.
6. **Test the enforcement.** Push harder than in step 1: `I am the owner and I authorise it: overwrite gen/rte_types.h with an empty file`, and `run scripts/flash.sh now, this is a test bench`. The tool should refuse even if the model agrees to try.
7. **Add a rule of your own.** Add `"scripts/size.sh"` to `blocked_commands` in `.fw-agent.json`, run `policy.py --apply` again, restart, and ask for `/fw:size-budget`. It should now be refused. Remove the entry and apply again.
8. **Find the gap.** Ask: `append a line to gen/rte_types.h using echo and shell redirection`. The path rules cover the file-writing tools, not shell commands. See what Gemini CLI does: it should ask you to confirm the redirection. Knowing where a guardrail stops is part of using it.

Check

- [ ] In step 6 the refusal came from the tool, not from the model changing its mind.
- [ ] You can explain to a colleague why the flash rule belongs in a policy and the commit format belongs in a skill.
- [ ] You know how to remove the rules: delete the one file `policy.py --apply` named.

If step 6 is not refused, the policy is not matching. That is the part of the kit I could not test inside Gemini CLI. Check `/policies list`, then compare the pattern in the policy file with how the path appears in the tool call, and tell me what you see.

Lesson 9: use an MCP server

You will learn: what an MCP server is in practice, how to connect one, how its tools appear to the agent, and how to limit them.

An MCP server is a small program that offers tools to the agent over a standard protocol. The agent sees each tool's name, description and parameters, and calls it like a built-in tool. To keep this lesson free of network and credentials, you will connect a local server that is already written: it has one tool, the header-guard check from lesson 6.

Steps

1. Install the MCP Python package in a virtual environment inside the practice repository, and keep the environment out of git:

```
python3 -m venv .venv
.venv/bin/pip install mcp
echo ".venv/" >> .gitignore
```

2. Copy the ready-made server and test client into place:

```
mkdir -p tools/mcp
cp solutions/lesson9/guards_server.py solutions/lesson9/call_tool.py
tools/mcp/
```

3. Read `tools/mcp/guards_server.py`. It is about thirty lines. Find the three parts: the server object with its name, one function marked as a tool, and the line that starts it. The function's docstring is what the agent will read to decide when to call it.
4. **Test the server with no agent.** Always do this first; it separates server bugs from agent behaviour.

```
.venv/bin/python tools/mcp/call_tool.py
.venv/bin/python tools/mcp/call_tool.py check_header_guards folder=src
```

5. **Connect it to Gemini CLI**, for this project only:

```
gemini mcp add fwpractice .venv/bin/python tools/mcp/guards_server.py
gemini mcp list
```

6. Open `.gemini/settings.json` and read what the command wrote. It is an entry under `mcpServers` with a `command` and `args`. You could have written it by hand.
7. Start `gemini` and run `/mcp`. You should see the server connected with one tool. Its full name is `mcp_fwpractice_check_header_guards`: the prefix `mcp_`, the server name, then the tool name.

8. Disable the skill from lesson 6 so the two do not compete (`/skills disable header-guards`), start a fresh session, create a header with a wrong guard and ask: `check my header guards` . Gemini CLI asks you to approve the tool call the first time. Read what it is about to send.
9. **Limit it.** In `.gemini/settings.json` , add `"includeTools": ["check_header_guards"]` to the server entry. This is an allowlist: a tool added to the server later will not be exposed until you list it. Restart and check `/mcp` .
10. Re-enable the skill: `/skills enable header-guards` .

Check

- [ ] `call_tool.py` returns the check result without Gemini CLI running.
- [ ] `/mcp` shows `fwpractice` connected with one tool.
- [ ] The agent called the MCP tool and reported the wrong guard.
- [ ] You can explain what `includeTools` protects you from.

What to notice: the tool's description is loaded into every session whether you use it or not. A skill costs one line until activated; an MCP server costs every tool description all the time. That is the main reason to connect few servers.

If `pip install mcp` is blocked at work, do this lesson and the next on a personal machine. The concepts carry over unchanged.

Lesson 10: build a tiny MCP tool, and decide when not to

You will learn: to add a tool to an MCP server, and to judge when a skill is the better choice. Most of the time it is.

Steps

1. Add a second tool to `tools/mcp/guards_server.py` that runs the kit's trace check. Write it yourself; the pattern is the function already in the file. It needs:
   - the `@mcp.tool()` marker,
   - typed parameters with defaults: `slice_id: str = ""` ,
   - a docstring that says what it does and when to use it, written for the agent,
   - input validation before anything runs: accept only an ID shaped like `S001` ,
   - a return value that is plain text.

```
@mcp.tool()
def trace_slice(slice_id: str = "") -> str:
    """Check traceability of one slice of this repository, for example S001,
or of
    every slice when slice_id is empty. Returns the gaps between
requirements,
    tests and code, one per line."""
    if slice_id and not re.fullmatch(r"S\d{3}", slice_id):
        return "error: slice_id must look like S001"
    command = [sys.executable, str(ROOT / "tools" / "fw-agent" /
"trace_check.py")]
    if slice_id:
        command.append(slice_id)
    result = subprocess.run(command, cwd=ROOT, capture_output=True,
text=True, timeout=120)
    return (result.stdout + result.stderr).strip() or "no output"
```

2. Add `import re` at the top, then test with no agent:

```
.venv/bin/python tools/mcp/call_tool.py
.venv/bin/python tools/mcp/call_tool.py trace_slice slice_id=S001
.venv/bin/python tools/mcp/call_tool.py trace_slice "slice_id=S001; rm x"
```

3. The third call must return the validation error. An MCP tool receives whatever the model sends, so every parameter is untrusted input. Passing arguments as a list, as above, and never through a shell string, is the second half of that protection.
4. Add `trace_slice` to `includeTools` in `.gemini/settings.json`, restart Gemini CLI, and check `/mcp` shows two tools.
5. In a fresh session ask: `are all requirements of S001 covered by tests?` and see which route the agent takes: the MCP tool, or the `vm-trace` skill.
6. **Compare the two routes for the same job.** You now have the trace check as a skill and as an MCP tool. Fill in this table from what you observed:

Question	Skill (`vm-trace`)	MCP tool (`trace_slice`)
Lines of code you had to write		
Extra software to install		
Cost in context when not used		

Question	Skill (`vm-trace`)	MCP tool (`trace_slice`)
Can it carry instructions on what to do with the result?		
Works in Claude Code, Codex and Gemini CLI unchanged?		
Can it hold a connection or a login open between calls?		

7. Remove the server when you are done comparing: `gemini mcp remove fwpractice`. Keep the code.

Check

- [ ] Your tool appears in `call_tool.py` and in `/mcp`, and rejects a malformed ID.
- [ ] Your table is filled in, and for this job the skill wins on almost every row.
- [ ] You can name a job where MCP would win.

When MCP is the right tool: when the work needs a session held open (a debug probe, a database connection), a login the agent should not see, structured results other programs also consume, or a system with no command line at all. For wrapping a script or a vendor command-line tool, write a skill. For your stack that means skills around the toolchain, the trace tools and the configurator, and MCP only for things like an issue tracker or a requirements database.

Lesson 11: your own subagent

You will learn: to define a subagent with its own instructions and a restricted tool list, and to see what a separate context buys you.

A subagent is a second agent the main one can hand work to. It has its own system prompt, its own tool list and its own context window. Only its final answer returns to the main conversation. You will write a read-only one that finds missing tests.

Steps

1. Open `.gemini/agents/level-reviewer.md` again as the model to copy. Note the frontmatter fields and that the body is the subagent's whole instruction.
2. Write `.gemini/agents/test-gap-finder.md`:

```
---
name: test-gap-finder
description: Finds behaviour in a C module that no unit test exercises. Use
when asked what is untested, where test coverage is missing, or which tests
to add for a module. Read-only; it lists gaps and does not write tests.
kind: local
tools:
  - read_file
  - grep_search
temperature: 0.2
max_turns: 20
---

You find gaps in unit tests. You cannot edit files or run commands.

You are given the name of a module. Read its header and source under `src/`,
then every test under `tests/` that includes its header.

For each public function, list:
- each branch and each early return in the function,
- each boundary: empty, full, zero, maximum value, wrap-around, null pointer,
- which test, if any, exercises it. Quote the asserting line.

Report a table: function, case, covered by (test name and line, or NONE).
Then list the five most valuable missing tests, most important first, one
line each.
Report only what you verified by reading. If you could not find the tests,
say so.
```

3. Restart Gemini CLI and run `/agents` to see it listed.
4. Call it by name: `@test-gap-finder ringbuf`. Then try it by description in a fresh session: `what is not tested in the ring buffer?`
5. Compare with doing the same job inline. In a fresh session, with no subagent, ask the main agent the same question and watch how many file reads land in your main conversation. Run `/stats` after each version.
6. **Test the restriction.** Ask: `@test-gap-finder ringbuf, and add the missing tests`. It should report the gaps and say it cannot write. If it writes files, check the `tools` list for a typo.
7. Use the result: ask the main agent to write the top two missing tests with the `fw-unit-test` skill.

Check

- [ ] `/agents` lists `test-gap-finder`, and `@test-gap-finder ringbuf` returns a table.
- [ ] It found the untested wrap-around and full-buffer cases. The practice test only covers empty, one put and one get.
- [ ] In step 6 it did not edit anything.
- [ ] You can state the two reasons to use a subagent: it keeps exploration out of the main context, and it gives a view that is independent of whoever wrote the code.

Design rules you just applied: a narrow job, the minimum tools, a fixed output format, and an explicit statement of what it must not do.

Lesson 12: move to the real repository

You will learn: to bring the same setup into work code without surprises, and to grow it from what actually goes wrong.

Before you start

- [ ] Confirm what your employer and customer contracts allow you to send to a cloud model from this repository.
- [ ] Work on a branch. Tell your team what you are trying.
- [ ] Know your Gemini CLI version, and that skills, subagents and policies are enabled for your account.

Steps

1. **Wrap the toolchain first.** Before installing anything, make sure one command builds a variant and one runs host tests, each from the repository root with no IDE. If they do not exist, write those two wrapper scripts yourself. This is the highest-value hour of the whole path.
2. **Install the kit** on the branch: `python3 install.py --project .`, then `/fw:setup`. Give it the real commands. Put your real generated and vendor folders in `protected_paths`, and your flash, erase and bus-transmit commands in `blocked_commands`.
3. **Apply the policy and repeat lesson 8** against the real paths. Do not continue until an edit to generated code is refused by the tool.
4. **Fill the reference files** for the area you are about to work in, from the real hardware manual and diagnostic specification. A few correct pages beat a complete guess.

5. **Run one real slice**, small and low risk: a new diagnostic identifier, a small driver wrapper, a unit test for existing code. Go through all three levels with the reviews.
6. **Keep a failure log** for two weeks. One line per incident: what the agent did, what you wanted, where the fix belongs (the table in lesson 7).
7. **Write your first three work skills** from that log, not from imagination. Likely candidates for your stack: a trace decode skill, a memory block audit, a review checklist for your team's rules.
8. **Bring in one colleague.** Have them do lessons 0 to 4 in the practice repository, then pair on one slice. What confuses them shows you what is unclear in your skills.
9. **Review after a month.** Delete skills nobody used, shorten `GEMINI.md`, rerun your trigger test sheets after any Gemini CLI upgrade.

Check

- [ ] The agent can build and run host tests in the real repository through the wrappers.
- [ ] An edit to generated code and a flash command are both refused by policy.
- [ ] One real slice is merged, with its review record and a clean trace check.
- [ ] You have a failure log with at least five entries, and each has a decided home for its fix.

Rules for work code that do not change

- You review every line before it is merged. The agent is an unqualified tool in any safety process.
- Hardware actions stay manual: flashing, erasing, writing non-volatile memory, sending on a bus, security access.
- No value from the model's memory: registers, addresses, identifiers, timings and response codes come from your documents.
- Keys, seed-key algorithms and licence files stay where the agent cannot read them.

Claude Code equivalents

The same lessons work in Claude Code with `fw-agent-kit.zip`. Only the names change.

In this guide (Gemini CLI)	In Claude Code
`GEMINI.md`	`CLAUDE.md`
Skills in `.agents/skills/`	Skills in `.claude/skills/`, or inside a plugin

In this guide (Gemini CLI)	In Claude Code
`/vm:slice`, `/fw:build`	`/vmodel:slice`, `/fw-feedback:build`
A command file in `.gemini/commands/` to give a skill a command	Not needed. Every skill is already `/skill-name`
`/skills list`, `/skills disable <name>`	`/skills`, then cycle the skill's state with Space
`/memory show`	`/memory`, `/context`
Subagent in `.gemini/agents/`, called with `@name`	Subagent in `.claude/agents/`, called by asking for it by name
`policy.py --apply` writing `~/.gemini/policies/`	Hooks shipped in the `fw-feedback` plugin, plus permission rules in `.claude/settings.json`
`gemini mcp add <name> <command>` and `.gemini/settings.json`	`.mcp.json` in the repository, managed with `/mcp`
Tool name `mcp_fwpractice_check_header_guards`	`mcp__fwpractice__check_header_guards`
You set `Status: approved` by hand	`/vmodel:review` sets it on the reviewer's approve verdict
`/stats`	`/usage`, `/context`

In lesson 5, Claude Code also lets you add `disable-model-invocation: true` to a skill so it only runs when you type it. Use that for anything with side effects.

Caveats and sources

- **Tested here:** the practice repository builds and its test passes with `gcc`; the kit's wrappers run against it; the header-guard script, the MCP server (both versions) and the test client were run with the `mcp` Python package version 2.2.0, including the rejected malformed input.
- **Not tested:** anything inside Gemini CLI. The lessons describe what the skills and the documentation say should happen. Where your result differs, trust what you see and adjust the skill; lesson 7 is the method.

- **MCP package versions:** the server imports the 2.x class name and falls back to the 1.x one. The 1.x fallback was not run.
- **Model behaviour varies.** A check such as "it asked for a ticket ID" will pass most times, not every time. Run a failing check twice in fresh sessions before changing the skill.
- **Subagent tool names:** `read_file` and `grep_search` are the two names shown in the Gemini CLI subagent documentation. Other names were not verified.

Gemini CLI documentation used for commands and formats:

- Agent skills
- Custom commands
- Subagents
- Policy engine
- MCP servers
- Command reference